COMPLEXITY ANALYSIS OF ALGORITHMS

Problem Solving with Computers-II

Join iclicker at **https://join.iclicker.com/GEXG**

Problem: Fibonacci Numbers

<u>Definition:</u>

The Fibonacci numbers are the sequence

1, 1, 2, 3, 5, 8, 13, 21, 34, 55,…

Defined by

$F_0 = F_1 = 1$ Base case $n = 0, 1$

$F_n = F_{n-1} + F_{n-2}$ for $n \geq 2$ $n > 1$

<u>Problem:</u> Given n, compute F_n.

Which implementation is significantly faster ?

A.

```
F(int n){
    if(n <= 1) return 1
    return F(n-1) + F(n-2)
}
```

Recursive

B.

```
F(int n){
    Initialize A[0 . . . n]
    A[0] = A[1] = 1

    for i = 2 : n
      A[i] = A[i-1] + A[i-2]

    return A[n]
}
```

Iterative

C. *Both are almost equally fast*

Which implementation is significantly faster ?

A.

```
F(int n){
   if(n <= 1) return 1
   return F(n-1) + F(n-2)
}
```

B.

```
F(int n){
    Initialize A[0 . . . n]
    A[0] = A[1] = 1

    for i = 2 : n
      A[i] = A[i-1] + A[i-2]

    return A[n]
}
```

C. *Both are almost equally fast*

The “right” question is: How does the running time grow?

E.g. How long does it take to compute F(200) recursively?

….let’s say on….a supercomputer that can compute 40 trillion operations per sec

How long does it take to compute Fib(200) recursively?

….let's say on…. a supercomputer that runs 40 trillion operations per second

It will take approximately 2^{92} seconds to compute F_{200}.

Time in seconds	Interpretation
2^{10}	17 minutes
2^{20}	12 days
2^{30}	32 years
2^{40}	35000 years (cave paintings)
2^{50}	35 million years ago
2^{70}	Big Bang

What is the main takeaway so far?

How long does it take to compute Fib(200) recursively?

….let's say on…. a supercomputer that runs 40 trillion operations per second

Algo A will take approximately 2^{92} seconds to compute F_{200}.

Analyze the algo

Big-O

theory

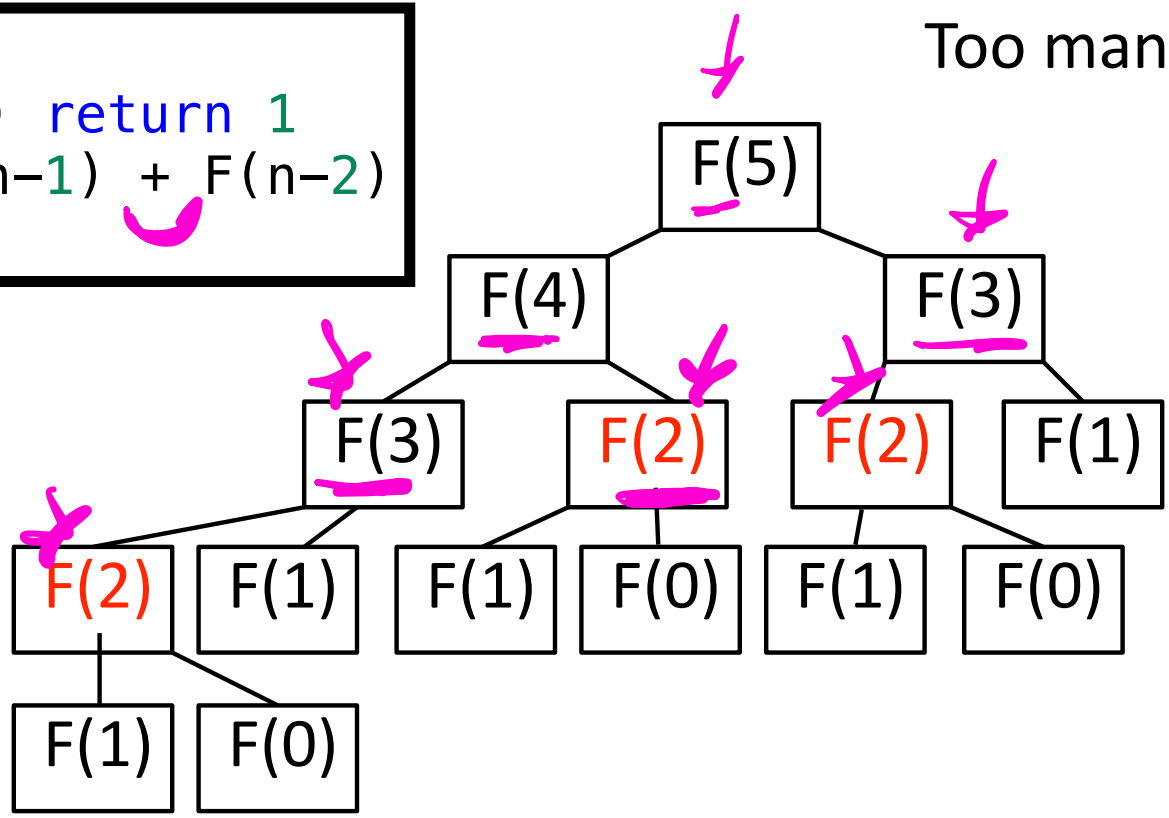
Questions of interest:

- Why is Algo A so slow?
- How do we quantify efficiency?
- Is Algo A better than Algo B?
- When will my code finish running?

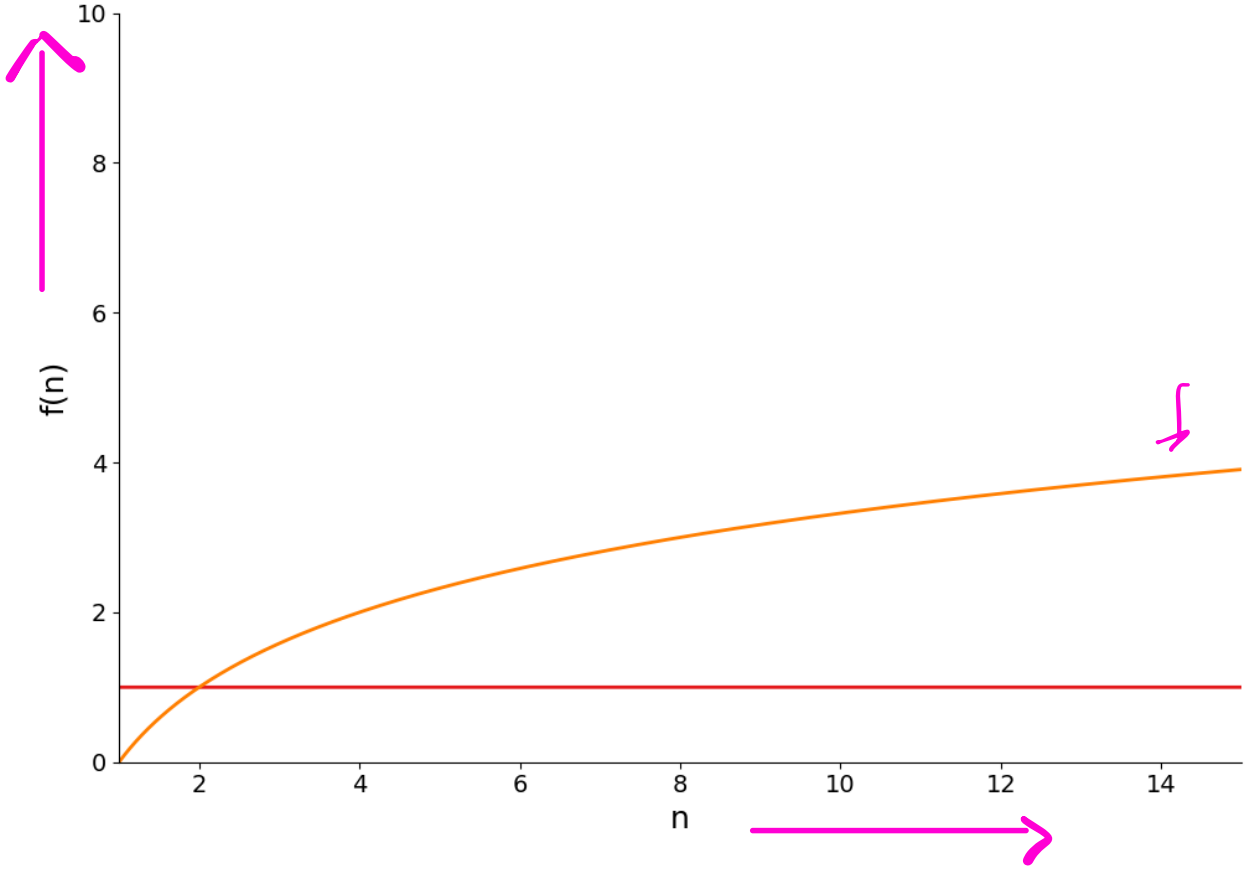
Why So Slow?

```
F(int n){
    if(n <= 1) return 1
    return F(n-1) + F(n-2)
}
```

Too many recursive calls.



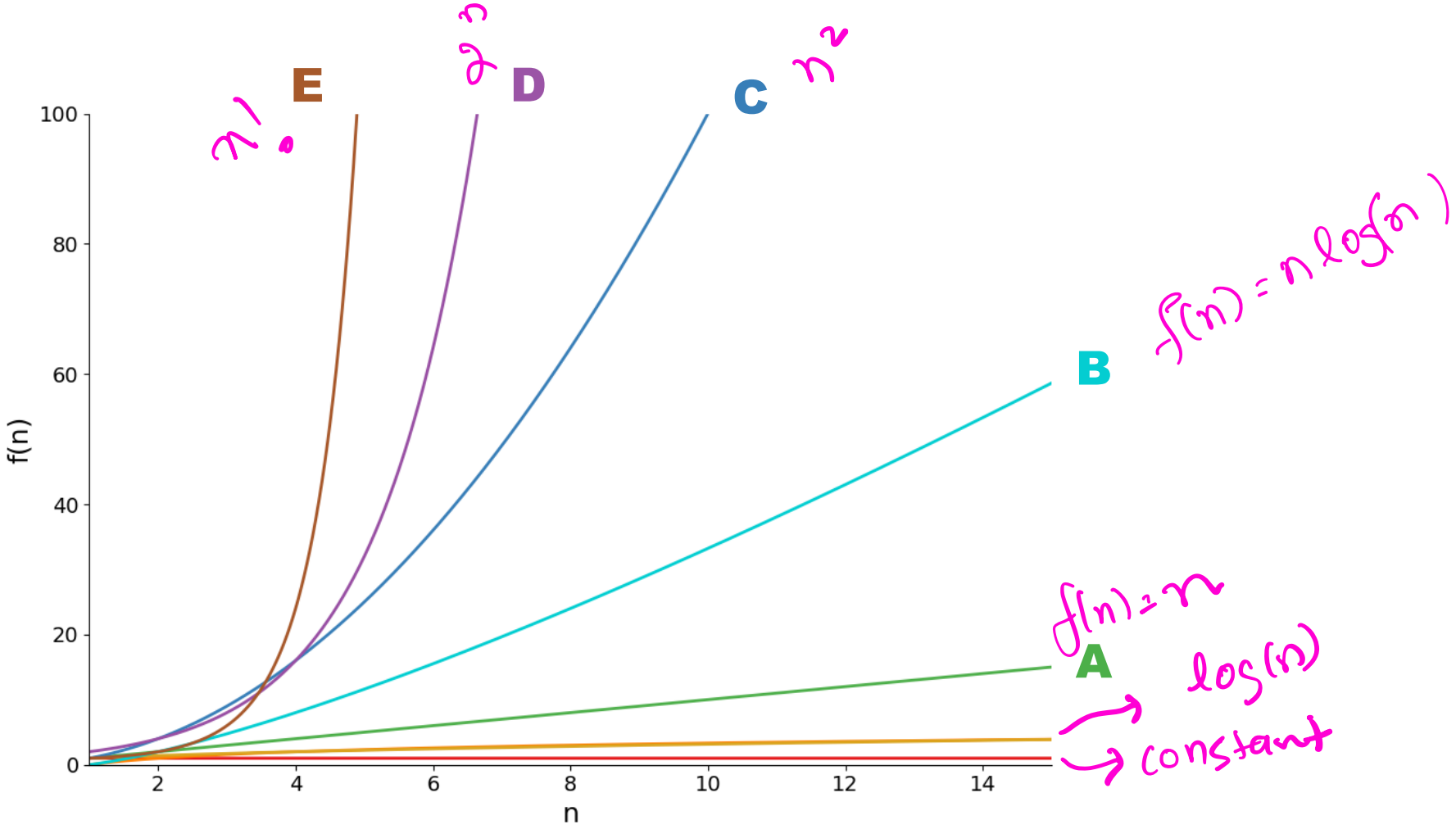
Identify the two functions in this plot.
Which curve represents how the running time of the recursive fibonacci algo grows?



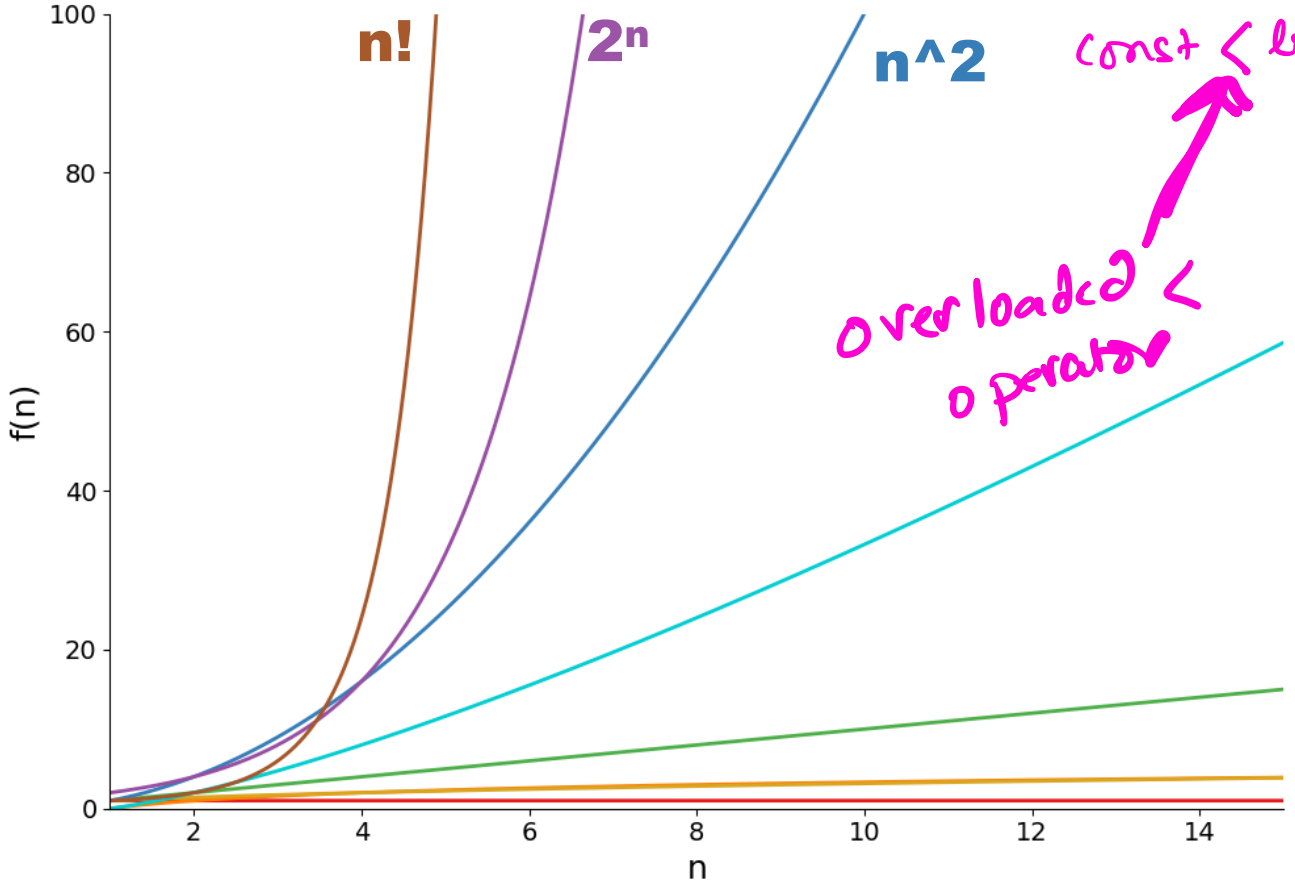
$f(n) = \log_2(n)$

$f(n) = \text{constant}$

Which curve represents how the running time of the recursive fibonacci algo grows?



- An **order of growth** is a set of functions whose growth behavior is considered equivalent.
- Functions that grow similarly belong to the same order of growth



const < log(n) < n < nlogn < n²
< 2ⁿ < n!

overloaded < operator

ORDERS OF GROWTH ACTIVITY

1. Rank these functions from SMALLEST to LARGEST growth order

$100 < n = 50n < 2n^2 \quad n \log n \quad 2^n$

$100 < n = 50n < n \log n < 2n^2 < 2^n$

2. Which functions belong to the SAME order of growth?

$n = 50n = O(n)$

3. The recursive Fibonacci has $O(2^n)$ order of growth.

4. The iterative Fibonacci has $O(n)$ order of growth.

Definition of Big-O

$g(n) = n^2$

f(n) and g(n) map positive integer inputs to positive reals.

We say f = O(g) if there is a constant c > 0 and k > 0 such that f(n) ≤ c · g(n) for all n >= k.

$f(n) = O(g(n))$

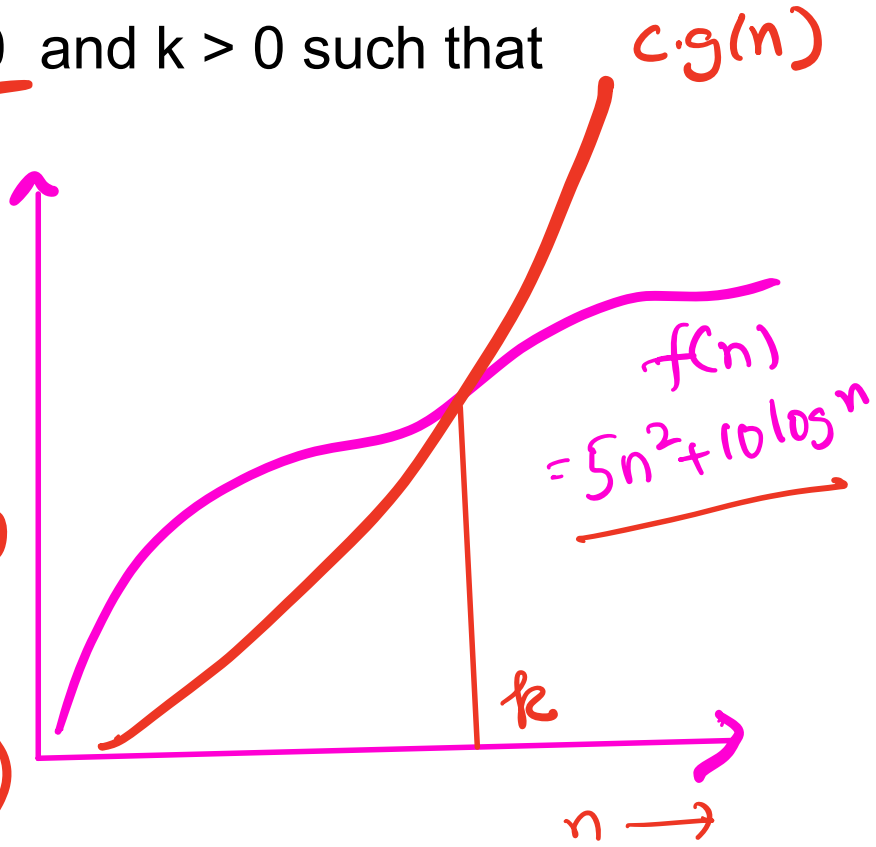
f = O(g)
means that "f grows no faster than g"

$f(n) \leq c \cdot g(n)$

for some const $c > 0$

for some const $k > 0$

$f(n) = 5n^2 + 10\log(n) = O(n^2)$



Big-O: Notation to name the order of growth

Order of Growth	Big-O Notation
Constant	$O(1)$
Logarithmic	$O(\log n)$
Linear	$O(n)$
Linearithmic	$O(n \log n)$
Quadratic	$O(n^2)$
Exponential	$O(2^n)$

- 50n and n are both O(n) — same order of growth.
- Big-O captures the growth rate, ignoring constants.

Express in Big-O notation

1. 10000000 $= O(1)$
2. 3n $= O(n)$
3. 6n-2 $= O(n)$
4. 15n + 44 $= O(n)$
5. 50nlog(n) $= O(n \log n)$
6. n^2 $= O(n^2)$
7. n^2-6n+9
8. $3n^2$+4*log(n)+1000 $= O(n^2)$
9. $3^n + n^3$ +log(3*n) $= O(3^n)$

$(n+1)(3n \log n + 100) = O(n^2 \log n)$

Common sense rules

1. Multiplicative constants can be omitted: $14n^2$ becomes n^2 .
2. n^a dominates n^b if a > b: for instance, n^2 dominates n.
3. Any exponential dominates any polynomial: 3^n dominates n^5 (it even dominates 2^n).

For polynomials, use only leading term, ignore coefficients: linear, quadratic

Big O analysis: Iterative Fibonacci

Running Time

Lets compute T(n) = **number of primitive operations** to execute **F**(n)

T(n) =

```
F(int n){
    Initialize A[0 . . . n]
    A[0] = A[1] = 1

    for (int i = 2; i <= n ; n++)
     A[i] = A[i-1] + A[i-2]

    return A[n]
}
```

$= O(n)$

$= O(1)$

$O(n)$

$O(1)$

Loop runs n times
For each iteration we do a const. amount of computation

$c_1 n + c_2$

$T(n) = O(n) + O(1) + O(n) + O(1) = O(n)$

Big O running time analysis: clicker

```
/* n is the length of the array*/
int sum(int arr[], int n)
{
      int result = 0;
      for(int i = 0; i < n; i+=2)
            result+=arr[i];
      return result;
}
```

number of times the loop runs $\frac{n}{2}$ times

$T(n) = 1 + 1 + 5 \cdot \frac{n}{2} + 1 + 1$

$= 4 + \frac{5n}{2} = O(n)$

A. $O(n^2)$
B. O(n)
C. O(n/2)
D. O(log n)
E. None of the above

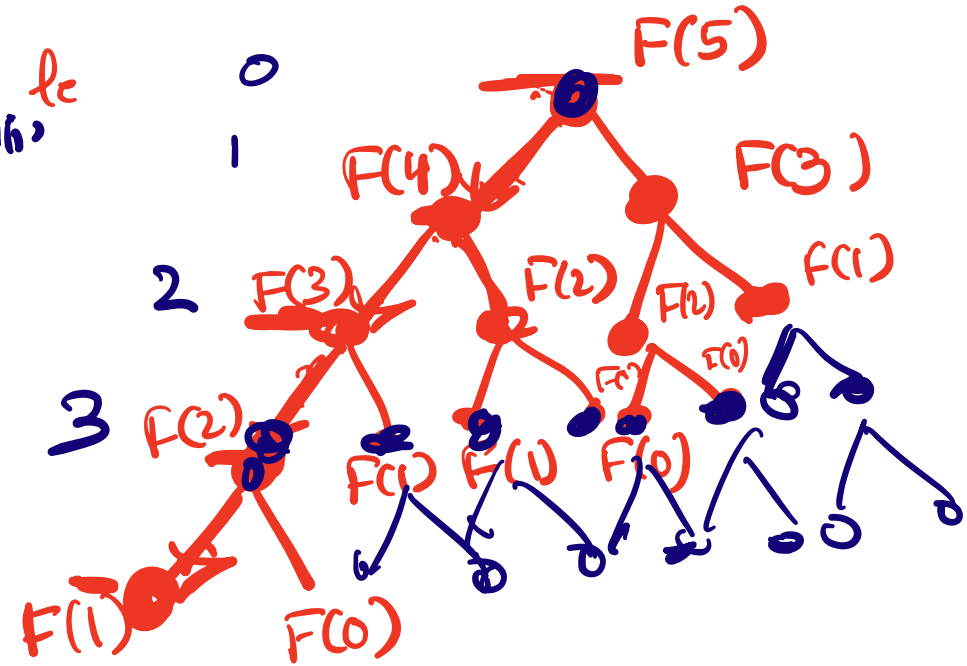
Derive $T(n) = O(2^n)$

```
F(int n){
    if(n <= 1) return 1
    return F(n-1) + F(n-2)
}
```

$T(n)$: Running time of $F(n)$

$T(n) = c \cdot$ (# of function calls)

level (i)	# of function calls at level i
0	1
1	2
2	4
3	8
4	16
⋮	⋮
(n-1)	2^{n-1}



Derive $T(n) = O(2^n)$

# of function calls

$$\leq 1 + 2 + 4 + 8 + \cdots 2^{n-1}$$

$$2^0 + 2^1 + 2^2 \cdots$$

$$= 2^{n-1+1} - 1$$

$$= 2^n - 1$$

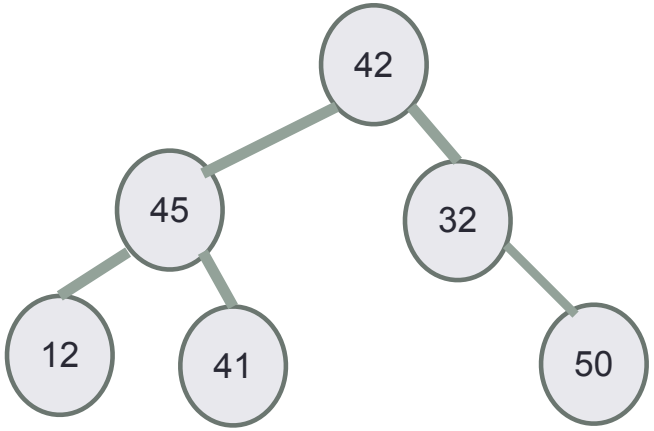
$$= O(2^n)$$

$$T(n) = c \cdot \#\text{ of calls}$$

$$= c \cdot O(2^n)$$

$$= O(2^n)$$

Binary Trees



In a tree, nodes are arranged in a hierarchy

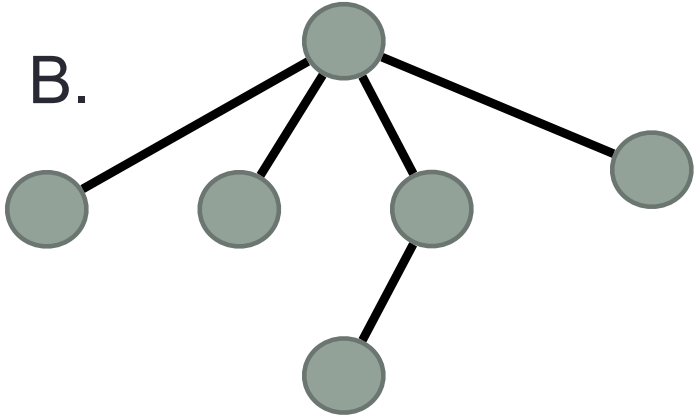
- One node is distinguished as the root
- Each node:
  - stores a key
  - has a pointer to child nodes and parent (optional)
- Unique path between any two nodes
- Leaf nodes have no children

In a binary tree, each node has at most two children

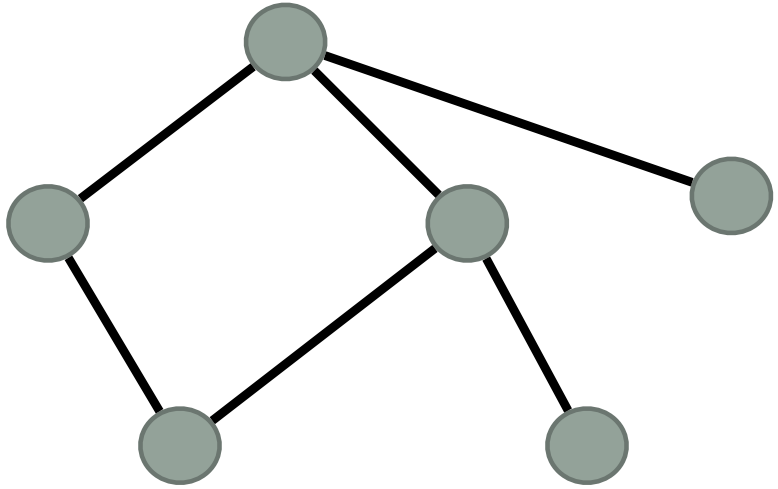
Which of the following is/are a tree?

A.

B.

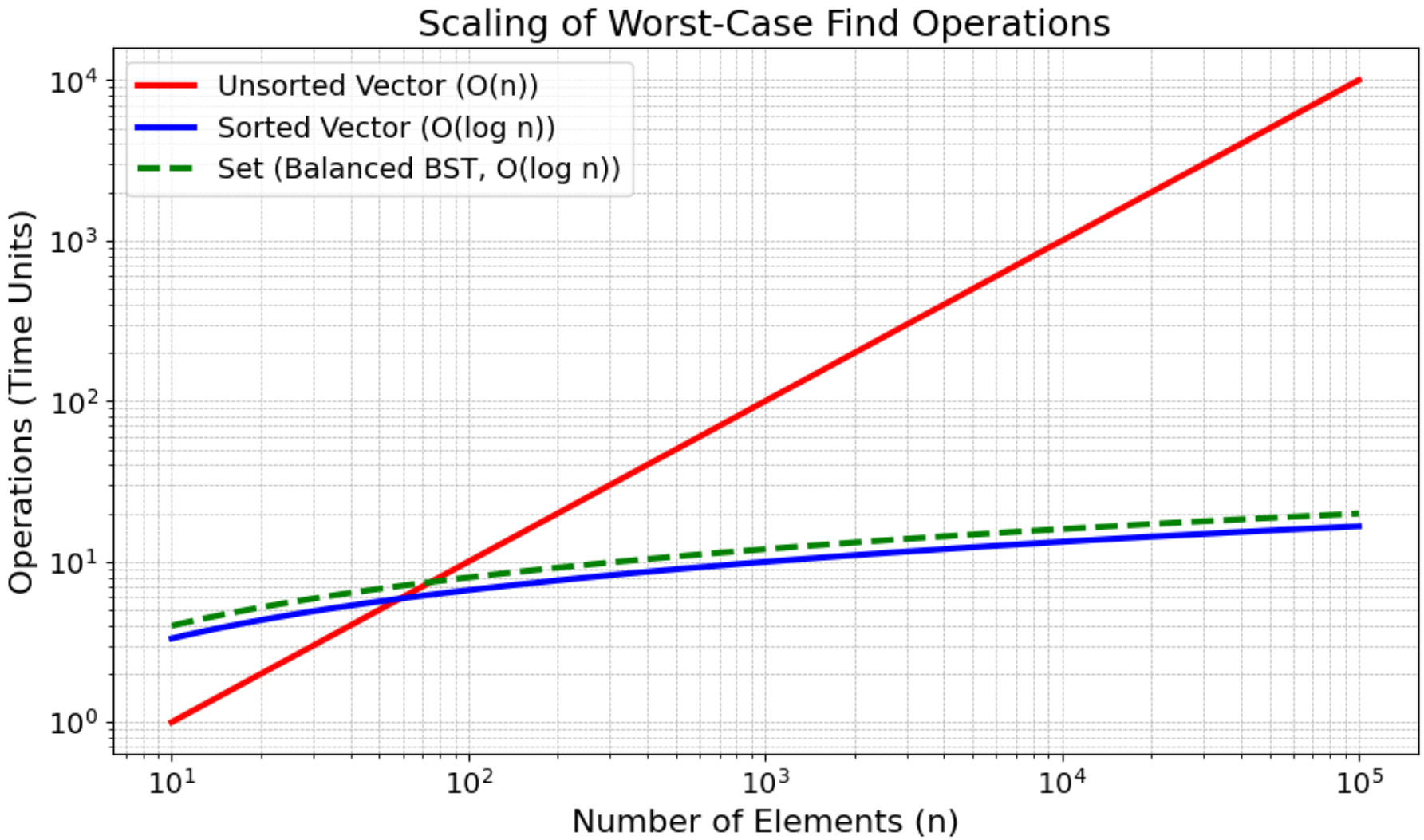


C.



D. A & B

E. All of A-C


Scaling of Worst-Case Find Operations
Unsorted Vector (O(n))
Sorted Vector (O(log n))
Set (Balanced BST, O(log n))
Operations (Time Units)
Number of Elements (n)
10^0
10^1
10^2
10^3
10^4
10^5

Next time

- Quiz 1: Includes Lecture 1 to 3 (up to rule of three).
- 35 minutes during lecture
- Bring dark pencil or pen
- Binary Search Trees

Credits and references:

Slides based on presentations by Professors Sanjoy Das Gupta and Daniel Kane at UCSD
https://cseweb.ucsd.edu/~dasgupta/book/toc.pdf